CACHE ENGINEERING FOR AGENTS

Tianyu Fu[*1], **Ruoxi Wang**[*1], **Zihan Min**[*1], **Zhongqi Ren**[1], **Zhihao Li**[1], **Tengxuan Liu**[1], **Shengxu Mu**[1], **Hanling Zhang**[2], **Yu Wang**[†1]

[1]Tsinghua University [2]The Chinese University of Hong Kong

arXiv:2610.28577v1 [cs.CL] 20 Oct 2026

ABSTRACT

Large Language Model (LLM) agents can handle increasingly complex tasks, navigating long trajectories of observations and actions. As these trajectories grow, key-value (KV) cache I/O and storage become the dominant cost. Trajectories can also exceed the LLM's maximum context length, requiring control over active cache growth. Prior work uses *context engineering* to reduce cache size by trimming or summarizing history. However, modifying the trajectory prefix invalidates subsequent cached states and triggers costly re-prefilling of retained context. Task accuracy can also suffer when too little context is retained to limit re-prefill costs. We propose *cache engineering* as a new scheme. It allows agents to directly control their active KV cache without modifying the trajectory or re-prefilling retained context. Specifically, agents can proactively use drop to exclude selected cache from future attention and repos to adjust retained keys' position encodings. We formalize these cache operations and identify history dependence as a serving challenge: identical token sequences can yield different cache states under different cache operation sequences, making prefix matching insufficient for correct cache reuse. Thus, we co-design the AKV serving engine. It provides a stateless interface compatible with existing serving APIs, while internally managing stateful cache reuse, eviction, and restoration. Experiments show that a tool-response rolling-drop cache policy improves average benchmark scores by 1.7 points and reduces average cost by 27.7%, compared with conventional threshold-triggered summarization. With AKV, it achieves up to $1.2\times$ the output throughput and a $1.5\times$ speedup in P95 request latency. Recursive self-improvement of cache policies further reduces cost by 7.1% over expert design. Our code is at `https://github.com/thu-nics/AKV`.

1 INTRODUCTION

Large Language Model (LLM) agents are capable of increasingly long-horizon tasks. Agents are generally composed of an LLM for reasoning and tool calling, and a harness that orchestrates tool execution and context. Through repeated reasoning, tool use, and environment interactions, they can carry out complex work spanning hundreds of rounds (Ye et al., 2025; Liu et al., 2025b).

As agent trajectories grow longer, Key-Value (KV) cache growth poses challenges to both inference cost and task accuracy. In terms of cost, retaining and repeatedly accessing accumulated KV caches incur substantial storage and I/O overhead (Wu et al., 2026; Anthropic, 2026). As shown in Figure 13, KV cache reads take up 82.4% of the total API cost. In terms of task accuracy, agents can suffer severe degradation if either the number of attended KV states or their positional range exceeds what LLM handles reliably (Hsieh et al., 2024; Chen et al., 2026b; Zhou et al., 2025). Therefore, it requires delicate control over the active KV cache for efficient and reliable agent execution.

Prior work reduces the growing cache via *context engineering*, typically by trimming or summarizing historical messages. While older messages are often less useful, editing them changes the trajectory prefix, invalidates subsequent KV states, and triggers re-prefill. The earlier the edit, the more retained trajectory suffix that must be re-prefilled. Common harnesses therefore defer reductions until an active context length threshold is reached, then trim or summarize most of the trajectory and

[*]Equal contribution. [†]Corresponding author: Yu Wang (yu-wang@tsinghua.edu.cn).

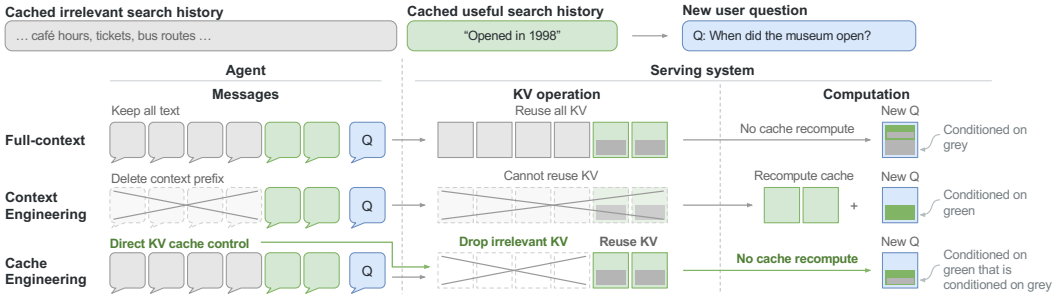


Figure 1: Full-context inference, context engineering, and cache engineering. Full-context inference reuses all historical KV. Context engineering removes irrelevant text and recomputes the retained history. Cache engineering keeps the text history but drops selected KV entries, reusing retained states without recomputation. Nested color blocks illustrate how retained KV can still carry information from dropped history.

retain only a few recent rounds to keep re-prefill short (Pi, 2026; OpenAI, 2026). Such aggressive reductions risk losing useful information from intermediate rounds, exposing a dilemma between re-prefill cost and task accuracy.

In this paper, we introduce *cache engineering*, a new scheme for agents to directly control their active KV cache. As shown in Figure 1, instead of modifying the text trajectory, it excludes only selected KV entries from future attention, and reuses all retained KV states without re-prefill. This allows agents to incrementally drop older KV entries while retaining more suffix KV within a given cost budget. To support this control, alongside the usual append operation that adds KV states for new tokens, we introduce two additional cache operations. Drop removes selected KV entries from future attention, and repos adjusts the positions encoded in retained keys. Throughout execution, the LLM and harness choose when and how to apply these operations, repeatedly interleaving them with append to shape the active cache. This programmable control also enables agents to search, evaluate, and refine cache engineering strategies through recursive self-improvement (RSI).

However, the history-dependent nature of cache engineering challenges current LLM serving systems. With standard KV caching, the token prefix fully determines the cached states (Kwon et al., 2023; Zheng et al., 2024). In contrast, the same token prefix and retained positions can result in different cache states under different cache operation histories. As shown in Figure 5, trajectories ① and ② both retain KV entries for B and C, but C's KV is computed with attention to A only in ①, due to the different ordering of drop and append operations. The serving system must therefore account for both token and cache operation histories to correctly track or reuse cache states.

Thus, we co-design AKV, a user-friendly and efficient serving system for cache engineering. It provides a *stateless* interface compatible with existing API-based serving, while handling the complexity of *stateful* cache management internally. Agents simply submit messages as usual, accompanied by cache operation instructions, without tracking which KV states remain resident on the server. The system uses an extended radix tree for correct reuse and timely eviction of history-dependent cache states, while segment-batch attention efficiently reconstructs missing states in a single prefill pass. AKV turns the theoretical efficiency potential of cache engineering into lower wall-clock latency and higher throughput for real-world agent workloads.

- **Cache engineering formulation.** We formalize cache engineering through explicit cache operations and their trajectories, providing a clear and deterministic description of history-dependent cache states. This formulation also provides a programmable interface for agents to define and refine their own cache engineering strategies.
- **Serving system for cache engineering.** We propose the first serving system that allows agents to directly control their caches, which were previously managed solely by the system. We co-design a stateless interface, an extended radix tree, and segment-batch attention to support efficient cache reuse, eviction, and restoration.
- **Improving efficiency and benchmark performance.** Across three LLMs and four agent benchmarks, a simple tool-response rolling-drop cache policy reduces average cost by

27.7% and improves average benchmark scores by 1.7 points, compared with conventional threshold-triggered summarization. With AKV server, this translates to $1.18\times$ the output throughput and a $1.51\times$ speedup in P95 request latency. An RSI-based policy additionally reduces cost by 7.1% on held-out tasks.

2 Related Work

Agent context engineering. At the text level, context engineering summarizes or trims interaction histories to reduce cost and enable agents to work beyond the LLM's context window. Existing methods trigger summarization through heuristics (Wu et al., 2025) or agent decisions (Ye et al., 2025; Sun et al., 2025). Further work improves summary quality through reinforcement learning (Lu et al., 2025) or by optimizing compression guidelines (Kang et al., 2025). To avoid the cost and complexity of summarization, trimming methods (Lindenbauer et al., 2025; Zhang et al., 2026) directly remove less relevant context. However, editing earlier context requires re-prefilling the retained suffix, creating a dilemma between re-prefill cost and information retention.

KV cache compression, eviction, and reuse. At the cache level, prior work reduces cache size or approximately reuses existing states to lower storage, access, and computation costs. *Compression* methods use low-rank representations (Liu et al., 2024a; Wang et al., 2024a), quantization (Liu et al., 2024b; 2026b), or aggregation (Fu et al., 2025b; Wang et al., 2024b) to represent cached entries more compactly. *Eviction* methods typically rely on low-level signals such as token positions (Xiao et al., 2024) or attention-based importance (Zhang et al., 2023; Li et al., 2024; Fu et al., 2025a; Yang et al., 2026) to remove less useful cache entries. *Reuse* methods adapt cached states to different positions or context prefixes through position encoding adjustments (Gao et al., 2024; Ma et al., 2026; Ordonez & Parnell, 2026) or partial recomputation (Wang et al., 2025). Our work focuses on eviction and reuse, enabling high-level eviction decisions by agents and harnesses, while providing a practical serving engine for efficient cross-request reuse and isolation. We discuss concurrent agent-oriented eviction work in Appendix A.5.

KV cache management in serving systems. Beyond algorithmic designs, previous works also explore system optimizations for efficient cache storage, sharing, and request scheduling. Existing systems use prefix caching to share KV states across requests with common prefixes while preserving equivalence to full prefill (Zheng et al., 2024; Ye et al., 2024). For agent workloads, recent systems batch context edits to reduce cache invalidation (Xu et al., 2026) or overlap re-prefill with ongoing execution to hide its latency (Pan et al., 2026). For cache engineering, it additionally requires tracking cache operation histories, since identical token prefixes can produce different states under different operations.

3 Cache Engineering

Cache engineering lets agents directly control their active KV cache without rewriting the trajectory or re-prefilling retained context. We introduce basic concepts in Section 3.1 and design motivations in Section 3.2, then formalize cache operations and their trajectories in Sections 3.3 and 3.4.

3.1 Preliminaries

An autoregressive LLM predicts the next token conditioned on the preceding token sequence $x_{1:t}$:

$$p_\theta(x_{t+1} \mid x_{1:t}). \tag{1}$$

Modern LLM serving uses KV caching to avoid recomputing earlier tokens' states. The next-token prediction is then conditioned on the current token x_t and the preceding tokens' cache:

$$p_\theta(x_{t+1} \mid x_t, \mathcal{C}_{1:t-1}), \tag{2}$$

where $\mathcal{C}_{1:t-1}$ denotes the KV states computed from tokens $x_{1:t-1}$.

An agent's *trajectory* is the ordered text history used for the following model computation, including its prompt, assistant messages, and tool responses. Formally, the trajectory is a *logical token sequence* $x_{1:t}$, and t is its *logical token length*.

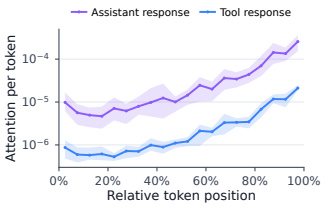


Figure 2: Attention over history. Per-token attention to assistant and tool responses, averaged over 10 BCP requests.

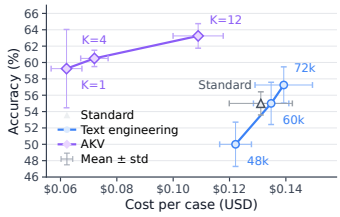


Figure 3: GPT-OSS-120B accuracy and cost on 100 BCP tasks. Labels indicate summary thresholds (tokens) or AKV's retained tool rounds.

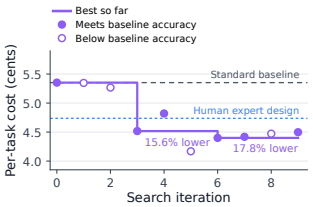


Figure 4: Recursive self-improvement on cache engineering policy, evaluated on the BCP benchmark with MiniMax-M2.7 LLM.

In cache engineering, the cache size need not equal the logical token length. We therefore define the *active cache* $\mathcal{C}$ as the KV cache states available to subsequent attention. The corresponding token sequence is the *active context*, whose length equals the number of token entries in $\mathcal{C}$. Note that *active* refers to participation in attention at the algorithmic level, not physical cache residency in the serving engine. A cache entry's *position encoding index* can also be adjusted independently of its token's *logical index* in the trajectory.

Context engineering modifies the logical token sequence through trimming or summarization. In contrast, *cache engineering* directly modifies the active cache through cache operations, while keeping the logical token sequence unchanged.

3.2 Design Motivation

The design of cache engineering stems from our thinking about three key questions: why direct cache modification is needed, which parts of the cache to retain, and how to manage its encoding positions. We discuss the motivations and empirical evidence guiding our design decisions below.

Why modify the cache directly. Cache engineering alleviates the dilemma between re-prefill cost and task accuracy in context engineering. We evaluate GPT-OSS-120B on the first 100 BCP tasks, with four trials per task and at most 100 agent rounds. Trim-based context engineering (retaining only the latest 12 rounds of tool responses) costs $1.85\times$ as much as the full-context baseline. Summary-based context engineering (48k threshold) reduces cost by about 7% but lowers accuracy by 5 percentage points (Figure 3). In contrast, directly dropping older tool-response cache can reuse the remaining cache, simultaneously improving accuracy and reducing cost (Figure 3). The retained cache may also preserve information from dropped history used in its computation (Xiao et al., 2024; Fu et al., 2026; Zou et al., 2025), as shown in Figure 1.

Which parts of the cache to retain. Cache selection should reflect message roles, recency, and task needs, beyond token-level importance heuristics. Figure 2 shows that older tokens receive less attention than recent ones, and tool responses receive less attention than assistant messages at comparable relative positions. We therefore use tool-response rolling drop as our primary example, retaining cache for all reasoning and recent tool responses while dropping older tool-response cache. We later explore alternative policies, including RSI-based cache engineering.

How to manage position encodings. By default, position encoding indices grow with the logical trajectory, even as the cache is reduced. Dropping cache neither changes retained position encodings nor resets the next token's position index, so indices can reach the model's limit even with a small cache. We decouple position encoding indices from logical indices and cache size, using a separate cache operation to reposition cached keys with RoPE rotations (Su et al., 2023) without re-prefill.

3.3 Cache Operations

We define three basic cache operations for cache engineering: append, drop, and repos, which directly manipulate cache without rewriting the existing logical token sequence.

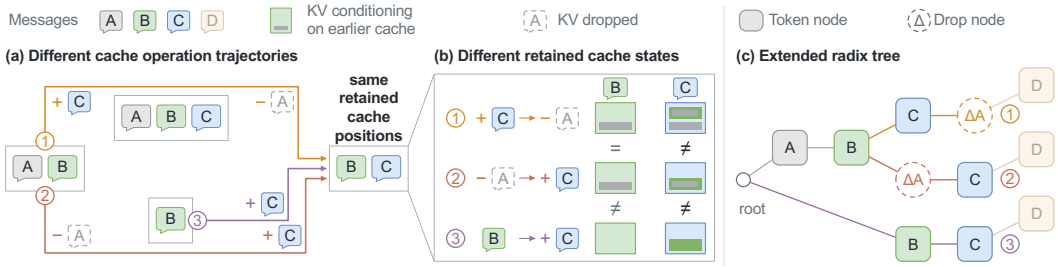


Figure 5: Cache operation trajectories and history-dependent KV states. (a) ① appends C to A and B, then drops A's KV; ② drops A's KV before appending C; ③ starts from B alone and appends C. All three retain B and C. (b) B's KV is identical in ① and ②, both conditioned on A, but differs in ③, where B has no prefix. C's KV differs in all three: ① is computed with A and B; ② and ③ cannot attend to A but use different B caches. (c) The extended radix tree shares the A–B prefix of ① and ②, then branches according to drop timing, distinguishing the cached states used to generate D.

Append. Operation append is the standard cache update in LLM inference. It processes a new token x_{t+1} conditioned on the active cache and appends its cache entry C_{t+1}:

$$\text{append}(x_{t+1}): \qquad \mathcal{C} \leftarrow [\mathcal{C}; C_{t+1}]. \tag{3}$$

We represent $\mathcal{C}$ as a sequence of KV entries ordered by their logical indices, where $[\cdot;\cdot]$ denotes concatenation. The token may come from the prompt, a tool response, or model generation. Position encoding indices start at zero and advance by one after each append; repos can reset the next index.

Drop. Operation drop removes selected KV states from subsequent attention, while leaving the retained states, their position encodings, and the logical token sequence unchanged (Figure 1):

$$\text{drop}(P): \qquad \mathcal{C} \leftarrow \mathcal{C} \setminus \{C_i : i \in P\}, \tag{4}$$

where P is the set of active logical token indices selected for dropping, and $\setminus$ denotes cache removal, preserving the order of retained entries. Note that drop does not change the position encoding indices used by subsequent append operations.

Repos. Operation repos updates cached keys' position encodings in logical token order, and resets the position index for subsequent append operations. With r retained cache entries, their position indices become $0, \ldots, r-1$, and the next appended token uses index r. For rotary position embeddings (RoPE) (Su et al., 2023), we move a cached key K from position index p to q by

$$K \leftarrow R(q)R(p)^{-1}K, \tag{5}$$

where $R(p)$ is the rotation matrix at position p. Cached values remain unchanged.

These operations define the cache visible to attention at the algorithmic level. The serving system handles physical storage, eviction, and reuse of cache separately in Section 4.

3.4 Cache Operation Sequence

To provide a clear and deterministic description of cache operations, we use a *cache operation sequence*:

$$o_{1:m} = (o_1, \ldots, o_m), \qquad o_j \in \{\text{append}(x), \text{drop}(P), \text{repos}\}. \tag{6}$$

Here, m counts operations, x is a token, and P contains the active logical token indices to drop. For a fixed LLM, this sequence determines the current cache states, starting from an empty cache.

Cache operation sequences are necessary because cache states are history-dependent: tokens and retained positions alone do not determine next-token probabilities. For example, in Figure 5, all three trajectories retain B and C, but A's cache is available when computing both B and C in ①, only B in ②, and neither in ③. Thus, C's cache states are different in all three trajectories. Similarly, repos changes the position encodings used by later append operations, as shown in Figure 8. Correct cache reuse and restoration therefore require both token and cache operation histories.

```
{  "model": "gpt-oss-120b",
   "messages": [
      {"role": "user", "content": "Search ICLR"},      ▷ append(m_0)
      {"role": "assistant", "content": "Searching"},  ▷ append(m_1)
      {"role": "tool", "content": "ICLR is..."}       ▷ append(m_2)
   ], "cache": {
      "drop": {"1": [0],                              ▷ drop(m_0)
               "2": [1]},                             ▷ drop(m_1)
      "reposition": [2]                               ▷ repos
} }
```

Figure 6: Illustrative AKV request. Right-hand annotations show cache operations; m_i denotes message i.

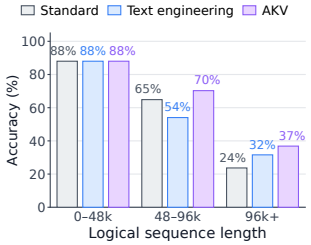


Figure 7: BCP accuracy by length with GPT-OSS-120B.

4 AKV Serving System

AKV follows the principle of *stateless interface, stateful serving*. Agents control the cache used in subsequent attention by specifying cache operations (Section 4.1), while the server handles physical cache reuse, eviction, and restoration internally (Section 4.2).

4.1 Serving Interface

The serving interface must keep each request's output distribution independent of physical cache residency and other requests' cache operations. For example, drop must affect only the issuing request's attention, leaving other requests' output distributions unchanged even when they share the same prefix cache. Each request must also fully describe the cache states it uses, without relying on physical cache retained from previous turns. This allows the server to fully restore missing cache states if memory pressure causes eviction between turns.

To meet these requirements, AKV extends the standard LLM serving API to specify both token and cache operation histories. As shown in Figure 6, the existing `messages` field provides the full logical token sequence, while the optional `cache` field specifies drop and repos operation sequence using an *operate-after* grammar. Note that all entries in `messages` implicitly specify append operations. Each key in `drop` identifies the message index after which to apply the operation; its value lists the message indices whose cache is dropped. The `reposition` field lists the message indices after which to apply repos. Message indices are zero-based, and drop precedes repos when both occur at the same message boundary. For multi-round agent workloads, clients extend the message and cache operation histories after each round, following the standard LLM serving workflow.

4.2 Cache State Management

4.2.1 Design Objectives and Challenges

AKV aims to preserve three key properties of conventional LLM serving systems: (1) reusing cached states if and only if they are equivalent to the required states, (2) evicting unused cache states without compromising request isolation, and (3) restoring evicted cache states within one prefill pass.

Serving systems face three corresponding challenges in preserving these properties when supporting cache engineering: (1) Token prefixes alone no longer determine cache states, so ordinary prefix matching can incorrectly reuse states from different cache operation histories (Figure 5(b)). (2) KV cache dropped by one request may still be needed by another, so evicting it without accounting for other requests can cause unnecessary recomputation or violate request isolation. (3) Ordinary causal prefill cannot reproduce cache states resulting from drops and repositions, while replaying the operation history requires multiple forward passes. AKV addresses these three challenges with an extended radix tree, drop-aware eviction, and segment-batch attention, respectively.

4.2.2 Drop-aware Cache Management

Reuse with drop nodes. AKV extends radix-tree prefix matching with drop nodes. When a request invokes drop, AKV inserts a drop node Δ at the trigger point along the request's path, recording the logical token indices whose cache is dropped. Because drop leaves retained states unchanged but

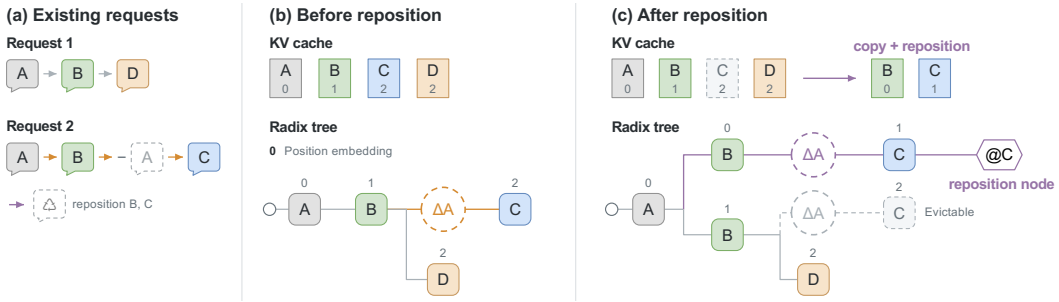


Figure 8: repos in the KV pool and radix tree. (a) Two requests share A and B; Request 2 drops A before appending C. (b) B and C initially occupy positions 1 and 2. (c) repos creates copies at positions 0 and 1 by re-encoding K and copying V. The original B remains shared with Request 1, while the original C becomes evictable. The tree branches at B to distinguish positional versions and records a reposition node after C. Labels p denote position p.

affects subsequent attention computation, this placement preserves prefix sharing while distinguishing later states. Prefix matching on the extended radix tree reuses cache along matched token and operation prefixes. In Figure 5(c), trajectories ① and ② share the A–B prefix but keep distinct C states because they drop A at different points, while trajectory ③ follows a separate branch. When processing D, each trajectory directly reuses its corresponding B and C caches.

Evict with drop-awareness. AKV uses drop information to reclaim unused cache before requests finish. A cache entry becomes evictable only when no ongoing computation or restoration needs it. For example, in Figure 5, A's cache can be freed once both requests ① and ② drop it, even as they continue generating D. Eviction frees the physical storage but leaves the tree node as an empty placeholder, keeping cached descendants accessible through prefix matching. Among evictable cache entries, AKV prioritizes evicting dropped cache over cache from completed requests, since the latter is more likely to be reused in the next round.

Restore with segment-batch attention. AKV uses *segment-batch attention* to restore missing cache states in a single prefill pass. At each layer, queries are split at drop trigger points, and each segment attends causally to its own tokens and to active cache from earlier segments. Note that dropped cache is excluded from later segments but remains available to earlier segments that originally used it, as shown in Figure 10. Only attention is segmented, while the FFN processes the full sequence as one batch. This avoids repeating FFN weight reads, a major source of memory traffic, for each segment.

4.2.3 Reposition-aware Cache Management

While drop leaves retained cache unchanged, repos efficiently modifies its keys' position encodings. Different requests may therefore need the same cache with different position encodings. Restoration must also reproduce the encodings used at each stage of computation.

Reuse with key rotation. AKV reuses cache after repos by copying cached states and rotating keys, without recomputing hidden states. In Figure 8(c), request 2 copies B and C from positions 1 and 2 to positions 0 and 1, while request 1 continues using the original B. In the radix tree, these copies form a new branch with updated position indices and a reposition node after C. Prefix matching checks both token and operation histories: matching position indices allow direct reuse, while differing indices require key rotation. Appendix A.1.2 explains why reposition nodes are necessary and details the matching rules and position transformations.

Evict positional versions independently. AKV evicts each position encoding versions independently, when it is no longer in use by any ongoing computation or restoration. For example, in Figure 8(c), the original B remains in use by Request 1, while the original C becomes evictable.

Restore with temporary KV versions. AKV extends segment-batch attention so each segment uses the position encodings from its original computation. Queries are also split at repos trigger points. At each layer, AKV rotates keys and copies values to create temporary KV versions for segments that require different positions, which we detail in Appendix A.1.1.

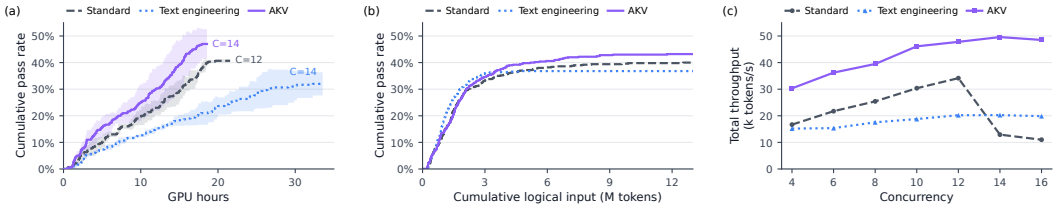


Figure 9: Task performance and serving throughput with GPT-OSS-120B on SWE-bench Verified. (a) Online evaluation on the first 100 tasks: mean cumulative pass rate versus GPU hours across three runs per method, with shaded regions indicating ± 1 standard deviation. (b) Cumulative pass rate versus cumulative logical input tokens over all 500 tasks. (c) Serving throughput versus concurrency.

5 Experiment

5.1 Setup

Baselines. We compare AKV with two context engineering baselines that summarize accumulated history, following common practice in agent harnesses (Anthropic, n.d.; OpenAI, n.d.). *Standard* uses a higher, model-specific summarization threshold: 96K tokens for GPT-OSS-120B and MiniMax-M2.7, and 192K tokens for MiroThinker-1.7-Mini. *Text Engineering* uses the same summarization policy with a lower threshold of 48K tokens across all models. For AKV, we use a tool-response rolling-drop policy that retains the KV states of the latest 12 tool responses. Its repos threshold matches Standard's summarization threshold for each model.

Models and benchmarks. We evaluate AKV across MiniMax-M2.7 (Chen et al., 2026a), GPT-OSS-120B (Agarwal et al., 2025), and MiroThinker-1.7-Mini (Team et al., 2026) LLMs. GPT-OSS-120B is evaluated with high reasoning effort, while MiniMax-M2.7 and MiroThinker-1.7-Mini are evaluated in thinking mode. The benchmarks cover various long-horizon agentic tasks, including BrowseComp-Plus (Chen et al., 2025) (BCP), xbench-DeepSearch-2510 (xbench, 2025) (xbench-DS), and the text-only 103-task validation subset of GAIA (Mialon et al., 2023) (GAIA-Text) for deep research, and SWE-Verified (OpenAI, 2024) for software engineering. Benchmark setup details are provided in Appendix A.2.1.

Metrics. We report success rate and mean per-task API cost for the main agent and its context-summary calls, in US cents unless otherwise specified. Costs are computed from active input and generated output tokens using Groq's rates (Groq, n.d.) for GPT-OSS-120B and MiniMax's official rates for MiniMax-M2.7. For MiroThinker-1.7-Mini, we use the official rates of its base model, Qwen3-30B-A3B (Yang et al., 2025). Detailed token prices are listed in Table 6. We additionally charge each token repos by AKV at the cache-hit rate, deliberately overestimating the cost of key rotation for a conservative comparison. For serving efficiency, we report request latency, time to first token (TTFT), and time per output token (TPOT), together with input and output throughput.

Serving configuration. Baselines run on SGLang (Zheng et al., 2024), which is also the base serving system of AKV. Serving experiments use eight NVIDIA A800-80GB GPUs: four for prefill and four for decode, with tensor parallelism of four. Since A800 GPU lacks native MXFP4 support, all LLM weights are set to BF16. Further details are in Appendix A.1.3.

5.2 Performance

Agentic benchmark results. As shown in Table 1, AKV achieves the highest average success rates across three models, improving the average success rate by 1.3–2.1% and 0.9–8.7% compared with two baselines, respectively. AKV also reduces inference cost by 27.7% relative to Standard and 21.9% relative to Text Engineering, averaged across all three models and four benchmarks. Appendix A.4.2 details the saving breakdown.

Source of accuracy improvements. AKV better maintains accuracy beyond the baselines' summarization thresholds. On the first 100 BCP tasks with GPT-OSS-120B, all three methods achieve comparable accuracy below 48K logical tokens (Figure 7). Text Engineering has the lowest accuracy in the 48–96K range, while Standard is lowest above 96K, following their respective 48K and

Table 1: Benchmark scores (pass rate, %) and costs (cents) by model and management method. Bold indicates the highest score and lowest cost within each model, including ties.

Method	BCP		GAIA-Text		SWE-Verified		xbench-DS		Average	
	Score	Cost	Score	Cost	Score	Cost	Score	Cost	Score	Cost
MiniMax-M2.7 (230B total, 10B active)										
Standard	58.9	5.2	35.0	**2.4**	**75.2**	7.0	**40.5**	5.5	52.4	5.0
Text Engineering	**62.3**	5.3	36.9	2.7	72.0	9.4	40.0	5.2	52.8	5.7
AKV	58.9	**4.8**	**40.8**	2.6	74.4	**5.5**	**40.5**	**4.7**	**53.7**	**4.4**
GPT-OSS-120B (5.1B active)										
Standard	57.5	13.1	68.0	5.4	40.0	20.0	**38.5**	6.2	51.0	11.2
Text Engineering	52.7	12.6	65.1	6.8	36.8	15.8	**38.5**	5.4	48.3	10.2
AKV	**61.9**	**11.4**	**69.9**	**4.1**	**43.2**	**9.7**	37.5	**3.8**	**53.1**	**7.3**
MiroThinker-1.7-Mini (30B total, 3B active)										
Standard	60.0	25.2	68.0	**9.2**	**7.8**	33.6	40.5	13.5	44.1	20.4
Text Engineering	45.7	17.6	66.0	11.0	4.2	28.4	32.5	15.3	37.1	18.1
AKV	**63.5**	**15.3**	**68.9**	9.5	7.4	**26.3**	**43.5**	**8.1**	**45.8**	**14.8**

Table 2: Serving performance (latency, s; TTFT, s; TPOT, ms/token; throughput, tokens/s) of GPT-OSS-120B on Top-80 SWE-Verified across concurrency (C) levels.

C	Method	Request latency		TTFT		TPOT		Throughput	
		mean	P95	mean	P95	mean	P95	Input	Output
8	Standard	12.16	43.57	3.18	8.42	26.57	31.38	25200	217
	Text Engineering	12.84	43.57	4.94	22.99	**23.23**	**25.68**	17313	228
	AKV	**7.79**	**25.93**	**1.87**	**5.99**	23.50	29.42	**39227**	**245**
10	Standard	12.82	45.03	3.93	9.76	26.47	30.83	30108	254
	Text Engineering	15.17	50.27	7.35	34.07	**22.57**	**25.09**	18533	246
	AKV	**8.13**	**27.11**	**2.22**	**7.36**	23.20	29.86	**46002**	**295**
12	Standard	12.39	43.16	2.97	9.99	28.00	32.78	33832	313
	Text Engineering	16.83	52.82	8.86	37.98	23.24	**25.95**	19945	264
	AKV	**8.47**	**27.18**	**2.60**	**8.17**	**23.06**	29.23	**48923**	**339**
14	Standard	30.16	108.86	21.36	88.29	**24.50**	29.85	12767	146
	Text Engineering	19.59	60.67	11.19	47.36	**24.50**	**27.23**	19992	267
	AKV	**9.09**	**28.49**	**2.92**	**8.62**	**24.50**	31.56	**51837**	**369**
16	Standard	36.50	229.90	27.71	223.57	24.83	29.25	10866	138
	Text Engineering	23.00	69.96	14.48	52.71	**24.44**	**27.56**	19595	267
	AKV	**10.32**	**30.63**	**4.16**	**13.02**	24.84	32.41	**50817**	**369**

96K summarization thresholds. This pattern is consistent with information loss from abrupt context reductions, whereas rolling drop removes older tool-response cache incrementally, as in Figure 14.

5.3 Efficiency

Serving efficiency. As shown in Table 2, AKV achieves higher throughput and lower request latency than both baselines across concurrency levels. At their respective peak-throughput settings, AKV ($C = 14$) achieves $1.36\times$ and $1.51\times$ speedups in mean and P95 request latency over Standard ($C = 12$), respectively. Input and output throughput are $1.53\times$ and $1.18\times$ those of Standard. Here, input throughput counts all logical input tokens, including those whose cache has been dropped. The workload and measurement protocol are detailed in Appendix A.1.4. AKV's throughput and latency

Table 3: Drop-target and repos ablations with GPT-OSS-120B on each benchmark's first 100 tasks.

Strategy	Repos	BCP		SWE	
		Score	Cost	Score	Cost
Standard	✗	56.0	13.5	**41.0**	19.1
Tool drop	✗	55.0	7.1	**41.0**	9.4
Reasoning drop	✓	56.5	8.2	36.5	14.2
Both drop	✓	56.5	8.7	34.5	7.8
Tool drop	✓	**62.0**	10.9	**41.0**	9.5

Table 4: Agentic cache reduction on a 100-task BCP subset. CAT means Cache-As-Tool.

Model	Metric	Method		
		Standard	Text Eng.	CAT
GPT-OSS-120B	Score	52.1	50.0	**61.0**
	Cost	13.4	13.9	**12.1**
MiniMax-M2.7	Score	56.0	61.0	**64.0**
	Cost	**5.4**	5.5	5.9

advantages over Standard widen at high concurrency ($C \geq 14$), as KV-cache pressure increases Standard's re-prefill work. As a result, fewer Standard requests can decode concurrently, yielding higher end-to-end latency and lower throughput.

Source of efficiency improvements. Figure 12 in Appendix A.4.1 shows that the decode speedup comes mainly from reduced full-attention time. Dropping older tool-response cache shortens the active context, reducing both KV reads and attention computation at each decode step.

Real-world task acceleration. In Figure 9(a), AKV reaches the highest final pass rate in the least GPU time. This online evaluation uses GPT-OSS-120B on the first 100 SWE-bench Verified tasks, with each method running at its peak-throughput concurrency. This advantage combines gains in task performance and serving efficiency. Figure 9(b) shows that AKV continues to improve its pass rate as cumulative logical input tokens increase, ultimately outperforming both baselines. Figure 9(c) shows higher peak serving throughput, allowing AKV to process these trajectories faster.

5.4 Ablations and Cache Engineering Policies

Ablation study. Table 3 compares drop targets and the use of repos, showing that tool-response drop with repos achieves the highest pass rate on both benchmarks. Adding repos raises BCP's pass rate from 55% to 62%, allowing exploration beyond the position limit that would otherwise halt execution. With repos enabled, dropping reasoning cache, either alone or together with tool-response cache, yields lower pass rates on both benchmarks than dropping tool-response cache alone.

Recursive self-improvement (RSI). Figure 4 shows that RSI finds lower-cost cache policies than our manually designed tool-response rolling-drop policy ($K = 12$) while maintaining Standard's accuracy. We use MiniMax-M2.7 to refine policies on 20 BCP tasks and evaluate the resulting policies on 100 held-out tasks. The policies from iterations 3 and 6 reduce per-task cost by 15.6% and 17.8% relative to Standard, respectively.

Cache management as a tool. Exposing cache management as a tool lets agents proactively decide when to reduce the cache and what to retain. Table 4 shows that this approach achieves the highest pass rate on a 100-task BCP subset for both models: 61.0% for GPT-OSS-120B and 64.0% for MiniMax-M2.7. Compared with Standard, GPT-OSS-120B's per-task cost falls from 13.4 to 12.1 cents, while MiniMax-M2.7's rises from 5.4 to 5.9 cents as it executes more turns.

6 Conclusion

We introduce cache engineering, enabling agents to directly manage active KV states while reusing retained caches without recomputation. We co-design the AKV serving system to support cache engineering through a stateless interface, an extended radix tree, and segment-batch attention for efficient cache reuse, eviction, and restoration. Across agent benchmarks, cache engineering improves average task performance while reducing inference costs, with AKV translating these efficiency gains into higher serving throughput and lower latency.

Limitations. Further exploration of cache engineering strategies, training agents under this scheme, and extending serving support to linear attention models remain future work.

ACKNOWLEDGEMENTS

This work was supported by National Natural Science Foundation of China (No. 62506197, 62325405, 62104128, U19B2019, U21B2031, 61832007, 62204164, 92364201), Tsinghua EE Xilinx AI Research Fund, and Beijing National Research Center for Information Science and Technology (BNRist). We thank all the support from Infinigence-AI.

REFERENCES

Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

Anthropic. Introducing claude fable 5.1 and claude mythos 5.1, 2026. URL https://www.anthropic.com/claude-fable-and-mythos-5-1.

Anthropic. How Claude Code uses prompt caching. Claude Code documentation, n.d. URL https://code.claude.com/docs/en/prompt-caching. Accessed September 22, 2026.

Aili Chen, Aonian Li, Baichuan Zhou, Bangwei Gong, Binyang Jiang, Boji Dan, Changhao Zhang, Changqing Yu, Chao Wang, Cheng Ma, et al. The minimax-m2 series: Mini activations unleashing max real-world intelligence. *arXiv preprint arXiv:2605.26494*, 2026a.

Guoxin Chen, Zile Qiao, Xuanzhong Chen, Donglei Yu, Haotian Xu, Wayne Xin Zhao, Ruihua Song, Wenbiao Yin, Huifeng Yin, Liwen Zhang, Kuan Li, Minpeng Liao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Iterresearch: Rethinking long-horizon agents with interaction scaling, 2026b. URL https://arxiv.org/abs/2511.07327.

Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhu Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025. URL https://arxiv.org/abs/2508.06600.

earendil-works. pi: AI agent toolkit: unified LLM API, agent loop, TUI, coding agent CLI. GitHub repository, 2026. URL https://github.com/earendil-works/pi. Commit ce5ec9ca355a852fb1f25c088cf409df47a95213; accessed September 26, 2026.

Tianyu Fu, Haofeng Huang, Xuefei Ning, Genghan Zhang, Boju Chen, Tianqi Wu, Hongyi Wang, Zixiao Huang, Shiyao Li, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. Moa: Mixture of sparse attention for automatic large language model compression, 2025a. URL https://openreview.net/forum?id=konDsSUSqg.

Tianyu Fu, Tengxuan Liu, Qinghao Han, Guohao Dai, Shengen Yan, Huazhong Yang, Xuefei Ning, and Yu Wang. Framefusion: Combining similarity and importance for video token reduction on large vision language models. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 22654–22663. IEEE, 2025b.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large language models. In *International Conference on Learning Representations*, volume 2026, pp. 43130–43158, 2026.

Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX annual technical conference (USENIX ATC 24)*, pp. 111–126, 2024.

Groq. Supported models. Groq documentation, n.d. URL https://console.groq.com/docs/models. Accessed October 15, 2026.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models?, 2024. URL https://arxiv.org/abs/2404.06654.

Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. Acon: Optimizing context compression for long-horizon llm agents. *arXiv preprint arXiv:2510.00615*, 2025.

Sanjay Kariyappa and G Edward Suh. Sidequest: Model-driven kv cache management for long-horizon agentic reasoning. *arXiv preprint arXiv:2602.22603*, 2026.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.

Junjie Li, Jiong Lou, and Jie Li. Intentkv: Cross-turn intent-aware kv cache pruning for agent inference. *arXiv preprint arXiv:2606.09916*, 2026.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.

Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as llm summarization for agent context management. *arXiv preprint arXiv:2508.21433*, 2025.

Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024a.

Junteng Liu, Yunji Li, Chi Zhang, Jingyang Li, Aili Chen, Ke Ji, Weiyu Cheng, Zijia Wu, Chengyu Du, Qidi Xu, Jiayuan Song, Zhengmao Zhu, Wenhu Chen, Pengyu Zhao, and Junxian He. WebExplorer: Explore and evolve for training long-horizon web agents, 2025a. URL https://arxiv.org/abs/2509.06501.

Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon swe-agents, 2025b. URL https://arxiv.org/abs/2512.22087.

Taowen Tony Liu, Jeffrey TH Wong, Can Xiao, Bowen Yang, Hao Mark Chen, and Yiren Zhao. Agentkv: Phase-aware kv eviction for agentic llms. *arXiv preprint arXiv:2609.14872*, 2026a.

Tengxuan Liu, Shiyao Li, Jiayi Yang, Tianchen Zhao, Feng Zhou, Xiaohui Song, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. Pm-kvq: Progressive mixed-precision kv cache quantization for long-cot llms. In *International Conference on Learning Representations*, volume 2026, pp. 107881–107900, 2026b.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024b.

Miao Lu, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen. Scaling llm multi-turn rl with end-to-end summarization-based context management. *arXiv preprint arXiv:2510.06727*, 2025.

Bole Ma, Jan Eitzinger, and Harald Köstler. Irminsul: Mla-native position-independent caching for agentic llm serving. *arXiv preprint arXiv:2605.05696*, 2026.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants, 2023. URL https://arxiv.org/abs/2311.12983.

OpenAI. Introducing SWE-bench Verified. OpenAI, 2024. URL https://openai.com/index/introducing-swe-bench-verified/. Updated February 24, 2025.

OpenAI. Unrolling the codex agent loop, 2026. URL https://openai.com/index/unrolling-the-codex-agent-loop/. OpenAI Engineering Blog.

OpenAI. Developer commands. OpenAI documentation, n.d. URL https://learn.chatgpt.com/docs/developer-commands?surface=cli. Accessed September 22, 2026.

Nathan Ordonez and Thomas Parnell. Minipic: Flexible position-independent caching in¡ 100loc. *arXiv preprint arXiv:2606.13126*, 2026.

Zaifeng Pan, Qianxu Wang, Zhengding Hu, Chang Chen, Yue Guan, Yanbo Zhou, Steven Swanson, and Yufei Ding. Smoothagent: Efficient long-horizon llm-based agent serving with lookahead context engineering. *arXiv preprint arXiv:2607.00151*, 2026.

Pi. Compaction reference, 2026. URL https://pi.dev/docs/latest/compaction. Accessed October 15, 2026.

Rulin Shao, Shannon Zejiang Shen, Junjie Oscar Yin, Yuetai Li, Minheng Wang, Hamish Ivison, Radha Poovendran, Nathan Lambert, Teng Xiao, Mike Lewis, et al. Context language models. *arXiv preprint arXiv:2609.37725*, 2026.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023. URL https://arxiv.org/abs/2104.09864.

Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.

MiroMind Team, S Bai, L Bing, L Lei, R Li, X Li, X Lin, E Min, L Su, B Wang, et al. Mirothinker-1.7 & h1: Towards heavy-duty research agents via verification. *arXiv preprint arXiv:2603.15726*, 2026.

Luning Wang, Shiyao Li, Xuefei Ning, Zhihang Yuan, Shengen Yan, Guohao Dai, and Yu Wang. Cskv: Training-efficient channel shrinking for kv cache in long-context scenarios. *arXiv preprint arXiv:2409.10593*, 2024a.

Qian Wang, Zahra Yousefijamarani, Morgan Lindsay Heisler, Rongzhi Gu, Bai Xiaolong, Shan Yizhou, Wei Zhang, Wang Lan, Ying Xiong, Yong Zhang, et al. Mepic: Memory efficient position independent caching for llm serving. *arXiv preprint arXiv:2512.16822*, 2025.

Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. Model tells you where to merge: Adaptive kv cache merging for llms on long-context tasks. *arXiv preprint arXiv:2407.08454*, 2024b.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025.

Yongtong Wu, Shaoyuan Chen, Yinmin Zhong, Rilin Huang, Yixuan Tan, Wentao Zhang, Liyue Zhang, Shangyan Zhou, Yuxuan Liu, Shunfeng Zhou, et al. Dualpath: breaking the storage bandwidth bottleneck in agentic llm inference. *arXiv preprint arXiv:2602.21548*, 2026.

xbench. xbench-DeepSearch-2510. Hugging Face dataset, 2025. URL https://huggingface.co/datasets/xbench/DeepSearch-2510. Accessed September 24, 2026.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pp. 21875–21895, 2024.

Buqiang Xu, Zirui Xue, Dianmou Chen, Chenyang Fu, Chiyu Wu, Caiying Huang, Chen Jiang, Jizhan Fang, Xinle Deng, Yijun Chen, et al. Tokenpilot: Cache-efficient context management for llm agents. *arXiv preprint arXiv:2606.17016*, 2026.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.15793.

Xintong Yang, Hao Gu, Binxing Xu, Lujun Li, Bei Liu, Jiacheng Liu, Qiyuan Zhu, Yike Guo, and Sirui Han. Indexmem: Learned kv-cache eviction with latent memory for long-context llm inference. *arXiv preprint arXiv:2605.25475*, 2026.

Lu Ye, Ze Tao, Yong Huang, and Yang Li. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11608–11620, 2024.

Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, et al. Agentfold: Long-horizon web agents with proactive context management. *arXiv preprint arXiv:2510.24699*, 2025.

Z.ai. GLM-5.2: Built for long-horizon tasks, June 2026. URL https://z.ai/blog/glm-5.2.

Haoxiang Zhang, Qixin Xu, Zhuofeng Li, Lei Zhang, Pengcheng Jiang, Yu Zhang, and Julian McAuley. Masking stale observations helps search agents–until it doesn't: A regime map and its mechanism. *arXiv preprint arXiv:2606.00408*, 2026.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025. URL https://arxiv.org/abs/2506.05176.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in neural information processing systems*, 36:34661–34710, 2023.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents, 2025. URL https://arxiv.org/abs/2506.15841.

Jiaru Zou, Ruizhong Qiu, Gaotang Li, Xiyuan Yang, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, et al. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.

A Appendix

A.1 AKV System Design Details

A.1.1 Restore with Segment-batch Attention

Segmentation. Building on Sections 4.2.2 and 4.2.3, AKV restores missing KV states by partitioning queries whenever a drop or repos operation changes the visible history or its position mapping. Each segment selects the corresponding KV pages and applies standard causal attention, encoding historical visibility without constructing a dense custom mask.

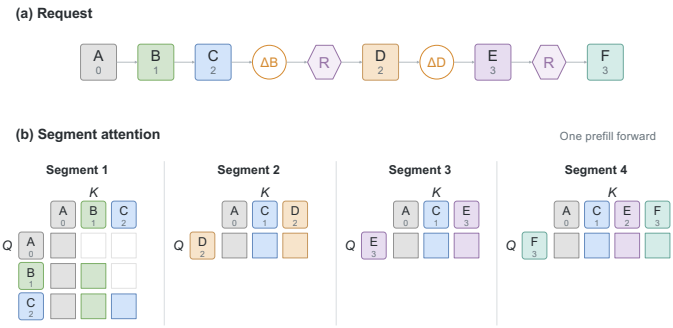


Figure 10: **Restore with segment-batch attention.** The request in (a) produces the four attention segments in (b). Numbers denote query/key positions; colored cells indicate allowed attention.

Example. Figure 10 illustrates a complete restoration, treating each letter as one token and writing X_p for token X at position p. Segment 1 computes A_0, B_1, C_2 with causal attention. After dropping B and repositioning, Segment 2 computes D_2 using keys (A_0, C_1, D_2). Dropping D starts Segment 3: E_3 attends to (A_0, C_1, E_3). Although Segments 2 and 3 share a position mapping, they require separate KV views because D is no longer visible to E; drop alone does not compact positions. The second repos moves E from position 3 to 2, so Segment 4 computes F_3 using (A_0, C_1, E_2, F_3).

Batched restoration. Within each layer, AKV writes the newly computed KV states and constructs the additional position versions, such as C_1 and E_2, by re-encoding K and copying V. The segment-specific page lists and query ranges are then submitted together to batched attention. Thus, all four segments execute within a single model forward pass, without recomputing hidden states for each position version or replaying separate forward passes at operation boundaries.

A.1.2 Reposition Scheme

Design objective. Reposition-aware cache management must distinguish positional versions while reusing their shared computational states. We therefore first identify directly reusable caches via exact matching, then use retrigger matching to find prefixes that are reusable via positional transformation, avoiding unnecessary hidden-state recomputation.

Token-node representation. Each token node is represented as $T(x, p)$, where x is the token ID and p is the position currently encoded in its cached K. The position label describes the stored cache, not necessarily the position at which the token was originally computed. For example, C is initially computed at position 2 in all three trajectories in Figure 11. After repos, trajectories 1 and 3 use $(C, 0)$, while trajectory 2 uses $(C, 1)$. Operation nodes along the path preserve the operation history that governs computation.

Why position labels and reposition nodes are needed.

Position labels distinguish cache representations that cannot be used interchangeably. For example, C has the same original computation history in trajectories 1 and 2, but is repositioned to 0 and 1, respectively. Its K encodings therefore differ and require positional transformation before reuse.

Reposition nodes distinguish states that share final positions but arise from different computations. Trajectories 1 and 3 both drop B after ABC, but trajectory 3 executes R_C before computing D.

When D is computed, the positions of A, C, and D are respectively $(0, 2, 3)$ and $(0, 1, 2)$. The relative distance between D and A changes, while that between D and C remains unchanged, potentially producing different hidden states and KV entries for D. Both trajectories subsequently drop A and execute R_D to move C and D to positions 0 and 1. This later transformation cannot erase the computational difference already established in D. The path must therefore retain R_C before D to prevent incorrect reuse of D and subsequent cached states.

Exact and retrigger matching. Exact matching compares all token-node fields:

$$T(x, p) \equiv T(x', p') \iff x = x' \wedge p = p',$$

and requires identical operation nodes and their parameters. Retrigger matching ignores only stored position encoding indices while preserving all operation records:

$$\phi(T(x, p)) = T(x), \qquad \phi(\Delta P) = \Delta P, \qquad \phi(R) = R.$$

Two prefixes are history-compatible if their projections are identical.

The system initializes the result with the longest valid exact prefix, then searches compatible branches from the root. A candidate replaces the current result only if it is valid and matches a strictly longer prefix, preserving exact matches in a tie. Retrigger matching stops at the first token or operation mismatch. Selected source caches are copied and positionally transformed as needed before use.

Correctness of retrigger matching. Under fixed model and position encoding configurations and exact arithmetic, identical projected prefixes preserve the token and operation history preceding each token's original computation. They therefore produce the same hidden states and unrotated KV content. Repos operations applied afterward change only K's positional representation, allowing a source version s to be transformed into a target version t as

$$K_t = R(p_t)R(p_s)^{-1}K_s, \qquad V_t = V_s,$$

without recomputing hidden states.

Preserving operation nodes prevents this transformation from crossing a divergence in computational history. For example, trajectories 1 and 3 disagree at R_C before D, so retrigger matching cannot reuse D across them. Finite-precision transformations may introduce rounding error; this correctness argument does not require bitwise equivalence to recomputation.

A.1.3 SERVING CONFIGURATION

AKV is implemented on SGLang v0.5.20. Table 5 summarizes the configuration used for the serving experiments. All three methods use GPT-OSS-120B with weights expanded to BF16 on eight A800 GPUs, with separate prefill and decode workers. Both workers enable radix caching; the baselines use LRU eviction, while AKV enables drop-aware eviction.

Table 5: **Serving configuration.**

Setting	Value
Model and weight precision	GPT-OSS-120B, BF16
Hardware	Eight NVIDIA A800 GPUs, 80 GB each
Parallelism	Prefill: TP4; decode: TP4
KV transfer	Mooncake over intra-node NVLink
Attention backend	Triton
MoE backend	`triton_kernel`
Static memory fraction	0.85
KV page size	One token per page
Prefill chunk size	8,192 tokens
Maximum prefill tokens	8,192
Prefix caching	Radix cache enabled on both workers
Eviction	Baselines: LRU; AKV: drop-aware eviction

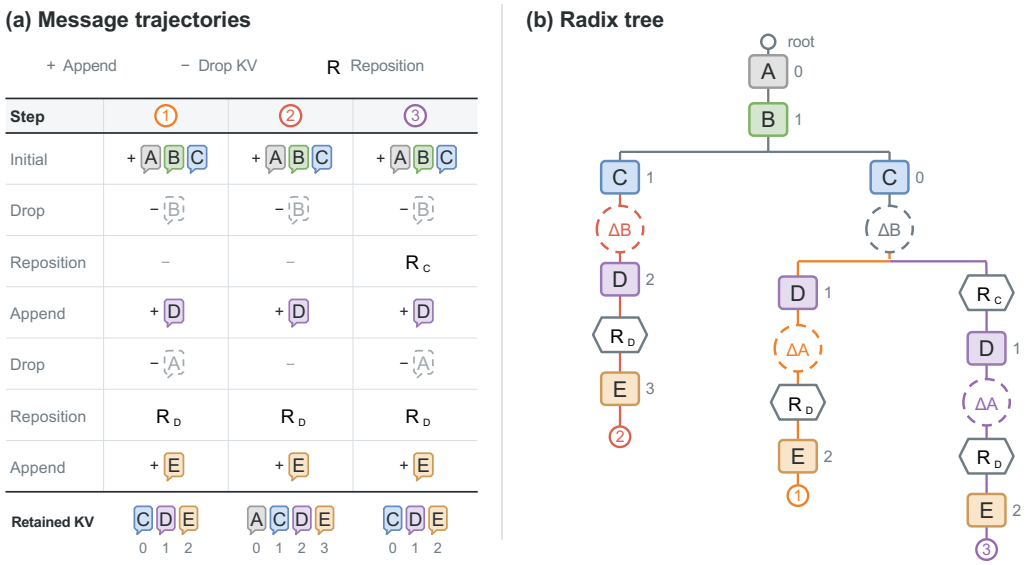


Figure 11: **Encoding drop and reposition histories in the extended radix tree.** (a) Three message trajectories interleave token appends, KV drops, and reposition operations. Although their retained tokens overlap, applying the operations at different points produces different KV states. (b) AKV records each operation in the radix tree, distinguishing cache versions by position and operation history. Numbers beside token nodes denote stored position encoding indices; R_X denotes repos after X.

A.1.4 Workload Replay and Evaluation Protocol

Task selection. To emulate realistic long-horizon agent workloads with substantial accumulated context, we select the 80 SWE-bench Verified tasks with the largest context lengths in trajectories, without any summary operation. These tasks reflect the difficulty of tasks in long-context reasoning. We rank tasks by this peak length and shuffle the selected tasks with a fixed seed. All methods use the same tasks and ordering. Each method replays trajectories collected under its own context/cache engineering policy, so the number of turns and the output budgets may differ between methods.

Trajectory replay. For each turn, we reconstruct the output token budget from the complete recorded message history, including reasoning and tool-call content, after subtracting the generation prefix already present in the input. We use temperature 0 and `ignore_eos=True` to generate the prescribed number of tokens. The newly generated assistant response is retained in the history for subsequent turns, while tool responses are replayed from the recorded trajectory rather than re-executing the tools. Recorded terminal inputs without an available assistant response are excluded from replay.

Concurrency and measurement windows. We maintain C client task concurrency and execute turns sequentially within each task. The task-count target for each measurement window is $\max(3 \times C, 30)$; measurements include requests from supplemental tasks used to maintain concurrency. For AKV at $C \in \{14, 16\}$, we truncate the archived runs at the completion of the first 30 tasks in completion order, counting all requests within the resulting window rather than only requests from those 30 completed tasks. For Standard at large concurrency, KV cache capacity pressure causes repeated prefill, resulting in severe serving degradation. We therefore report these two runs over a fixed one-hour window, with some requests still unfinished at the cutoff.

A.2 Experimental Details

A.2.1 Evaluation Configurations

Evaluation harnesses. We adapt the official BrowseComp-Plus harness, mini-SWE-agent (Yang et al., 2024), the browser-agent scaffold from observation-masking (Zhang et al., 2026), and WebExplorer's evaluation harness (Liu et al., 2025a) for BCP, SWE-bench Verified, GAIA-Text, and xbench-DeepSearch-2510, respectively.

Retrieval and webpage reading. BCP evaluation follows the official task prompts and uses Qwen3-Embedding-8B (Zhang et al., 2025) to retrieve documents from the benchmark's fixed corpus. All models and methods have access to both `search` and the official optional `get_document` tool, which returns the full text of a selected document. For xbench-DeepSearch-2510, we use GLM-5.2 (Z.ai, 2026) to extract information from webpages in response to the agent's queries.

Answer grading. We use GLM-5.2 as the answer judge for all three question-answering benchmarks. The grading prompts follow the official BCP implementation for BCP, observation-masking for GAIA-Text, and WebExplorer for xbench-DeepSearch-2510. For SWE-bench Verified, submitted patches are evaluated using the official test scripts and grading functions; no LLM judge is used.

Agent step limits. The agent step limits are 100 for BCP, 250 for SWE-bench Verified, 200 for GAIA-Text, and 100 for xbench-DeepSearch-2510.

A.2.2 Context Engineering Baseline Details

Our context engineering baselines adopt Pi's summarization prompts, conversation serialization, cut-point selection, and summary-generation budgets (earendil-works, 2026), with `reserveTokens`=16,384 and `keepRecentTokens`=20,000 across experimental groups. Summarization uses the agent's model and reasoning settings, replacing older dialogue with a summary while preserving system and developer instructions and the selected recent messages.

Unlike Pi's native trigger based on the context window minus the reserve-token budget, we trigger compaction when the exact prompt token count reaches the threshold specified in Section 5.1. We also reject empty summaries and handle summarization failures according to the benchmark error protocol without retry.

Pi defines four prompt templates: a shared system prompt, an initial-summary prompt, an update-summary prompt, and a turn-prefix prompt. For brevity, we reproduce the shared system and turn-prefix prompts below. The turn-prefix prompt is appended to the serialized history in the summary request's user message.

Text 1. **Shared system prompt**

```
You are a context summarization assistant.  Your task is to read a
conversation between a user and an AI assistant, then produce a
structured summary following the exact format specified.

Do NOT continue the conversation.  Do NOT respond to any questions
in the conversation.  ONLY output the structured summary.
```

Text 2. **Turn-prefix prompt**

```
This is the PREFIX of a turn that was too large to keep.  The
SUFFIX (recent work) is retained.

Summarize the prefix to provide context for the retained suffix:

## Original Request
[What did the user ask for in this turn?]

## Early Progress
- [Key decisions and work done in the prefix]

## Context for Suffix
- [Information needed to understand the retained recent work]

Be concise.  Focus on what's needed to understand the kept suffix.
```

A.3 API Cost Accounting for Experiment

We estimate API-equivalent inference costs using the fixed token prices listed in Table 6. Our cost accounting includes only the evaluated agent's generations and its context-summary calls. We use Groq's rates (Groq, n.d.) for GPT-OSS-120B and GPT-OSS-20B, and MiniMax's official rates for MiniMax-M2.7. For MiroThinker-1.7-Mini, we use the official rates of its base model, Qwen3-30B-A3B (Yang et al., 2025). Cache-miss, cache-hit, and output tokens are charged separately at different rates, while using the same prices across different methods for the same model. For AKV's cache repositioning, we also charge it at the cache-hit rate as a conservative estimate of the cost of key rotation.

Table 6: Token prices for API-equivalent cost rate (USD per million tokens).

Model	Cache-miss	Cache-hit	Output
MiniMax-M2.7	0.300	0.060	1.200
GPT-OSS-120B	0.150	0.075	0.600
MiroThinker-1.7-Mini	0.200	0.040	2.400
GPT-OSS-20B	0.075	0.0375	0.300

A.4 Additional Experiment Results

A.4.1 Speedup Breakdown

AKV accelerates decoding primarily by reducing full-attention time. As shown in Figure 12, at the same batch size, AKV reduces full-attention time by 59.8% and total decode-step time by 16.0% relative to Standard. Dropping older tool-response KV shortens the active context, so subsequent decode steps load less KV data from GPU memory and perform fewer attention computations.

A.4.2 Cost Breakdown and Active Context Length

Cost breakdown. AKV lowers costs by reducing cache reads and repeated prefill. As shown in Figure 13, dropping older tool-response KV reduces the amount of cached history read by subsequent requests, lowering cache-read costs by approximately 17% relative to Standard. Compared with Text Engineering, AKV reuses retained KV directly without the re-prefill required when summarization rewrites the history. This reduces re-prefill computation and lowers cache-miss input costs by approximately 48%.

Active context length. Figure 14 shows how AKV slows the growth of active context. Once tool responses exceed the retention budget of K rounds, AKV continually drops older tool-response KV while retaining the most recent K rounds and reasoning KV. The active context therefore continues to grow as reasoning accumulates, but at a substantially slower rate. In contrast, Standard accumulates history until summarization triggers a sharp reduction in context length.

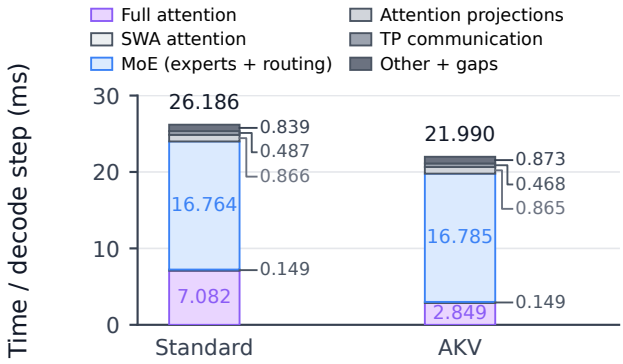


Figure 12: **Latency breakdown.** Decode-step time for GPT-OSS-120B at the same batch size, broken down by operation.

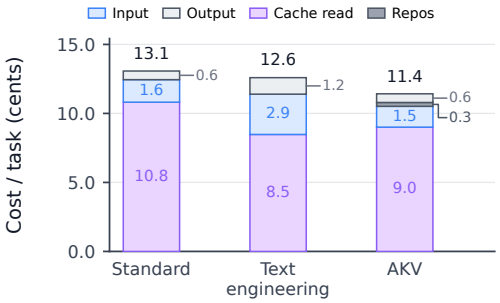


Figure 13: **Cost breakdown.** Average cost per task on BCP with GPT-OSS-120B for Standard, Text Engineering, and AKV.

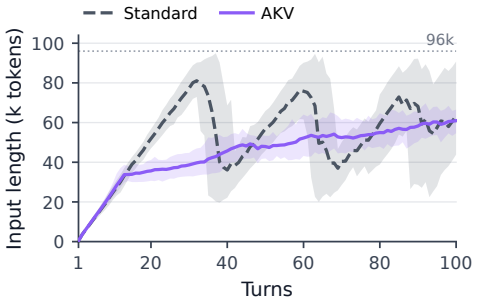


Figure 14: **Active context length by turn.** Standard and AKV on BCP over 100 turns. The dotted line marks 96k tokens.

A.4.3 Attention Across Message History

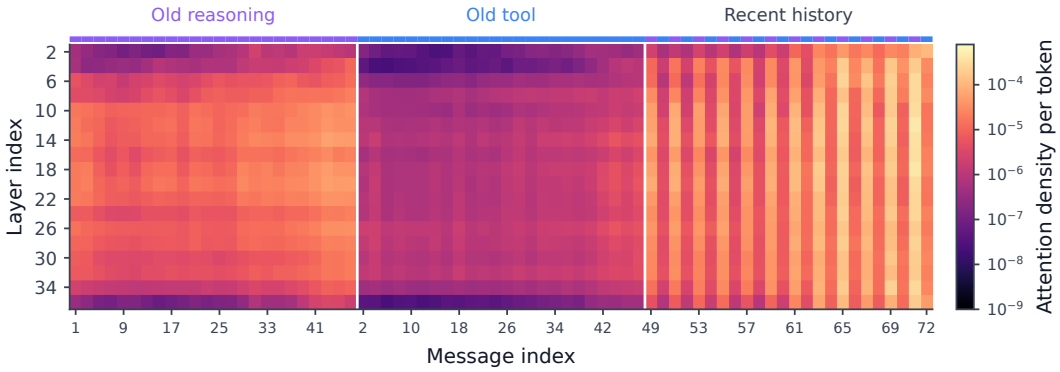


Figure 15: **Attention density across message history.** Attention density per token by layer and message index averaged over 10 selected BrowseComp-Plus trajectories, divided into old reasoning, old tool responses, and recent history.

To explain why our tool-response rolling-drop policy prioritizes dropping older tool-response KV entries, we analyze the attention allocated to different types of historical messages. Figure 15 shows attention per token across layers averaged over 10 BrowseComp-Plus trajectories. Older tool responses receive less attention than older reasoning in most displayed layers, while recent history

generally receives more attention. This pattern suggests that direct attention to historical messages depends on both recency and message type, providing empirical support for retaining recent tool responses and reasoning while preferentially dropping older tool-response KV entries. Together with the task performance and cost results reported earlier, this observation helps explain how AKV reduces KV-cache access while maintaining task performance.

Compatibility with Harness. To assess whether AKV improves an existing harness, we integrate it with Pi (earendil-works, 2026) and evaluate the combination on BCP. For both GPT-OSS-120B and GPT-OSS-20B, adding AKV improves accuracy by 3% over the native Pi harness without compaction and 4% over the Pi harness with compaction.

A.4.4 Additional Models

Table 7: Benchmark scores (pass rate, %) and costs (cents) of GPT-OSS-20B across different management methods. Bold indicates the highest score and lowest cost, including ties.

Method	BCP		GAIA-Text		SWE-Verified		xbench-DS		Average	
	Score	Cost	Score	Cost	Score	Cost	Score	Cost	Score	Cost
Standard	49.0	6.5	60.2	3.5	41.8	12.5	**25.0**	2.4	44.0	6.2
Text Engineering	43.0	**4.4**	**65.1**	3.0	25.6	**8.8**	19.0	2.0	38.2	**4.6**
AKV	**50.5**	6.0	62.1	**2.5**	**44.2**	8.9	24.0	**1.8**	**45.2**	4.8

We further evaluate AKV on GPT-OSS-20B (Agarwal et al., 2025), a smaller model from the same model family as GPT-OSS-120B, and compare its pass rate and cost against the Standard and Text Engineering baselines. GPT-OSS-20B is evaluated with high reasoning effort across all methods. We use the same thresholds as for GPT-OSS-120B: 96K tokens for Standard summarization and AKV's repos, and 48K tokens for Text Engineering. As shown in Table 7, AKV achieves the highest average pass rate of 45.2%, while maintaining a comparable average cost of 4.8 cents. In particular, AKV achieves the best pass rate on BCP and SWE-Verified, and the lowest cost on GAIA-Text and xbench-DS. On several metrics where AKV does not outperform the baselines, we attribute the smaller gains to the more limited capability of GPT-OSS-20B: compared with GPT-OSS-120B, the smaller model is less able to effectively leverage the retained information under cache engineering, making it more sensitive to cache modifications.

A.5 Additional Related Works

Concurrent works. Concurrent work adapts KV eviction to the changing attention needs of agent trajectories. AgentKV (Liu et al., 2026a) scores KV entries using query buffers from different execution phases, retaining information that recency-based scoring can miss. IntentKV (Li et al., 2026) combines cross-turn query memory with learned importance estimates to preserve useful history under limited KV budgets. SideQuest (Kariyappa & Suh, 2026) explores model-directed retention through an auxiliary reasoning branch that identifies stale tool results and issues cache-deletion commands, but its current strategy is limited to tool responses, and its serving integration is tailored to this specific eviction workflow. Context Language Models (CLMs) (Shao et al., 2026) primarily study model-driven context management, treating the live context as an editable file and learning how to maintain it. CLM also introduces Suffix Cache Reuse to preserve unchanged downstream KV states after text edits. Its serving support centers on session-local reuse: relocated states remain in private buffers outside the shared radix cache, without a unified mechanism for indexing, managing, and recovering these history-dependent states. AKV addresses these serving concerns directly by exposing policy-independent cache operations through a stateless interface and uses an extended radix tree to track both tokens and cache operation histories, integrating retained states into shared cache reuse, memory management, and recovery.